XỬ LÝ ẢNH SỐ · CHỦ ĐỀ 4 – PROJECT 2

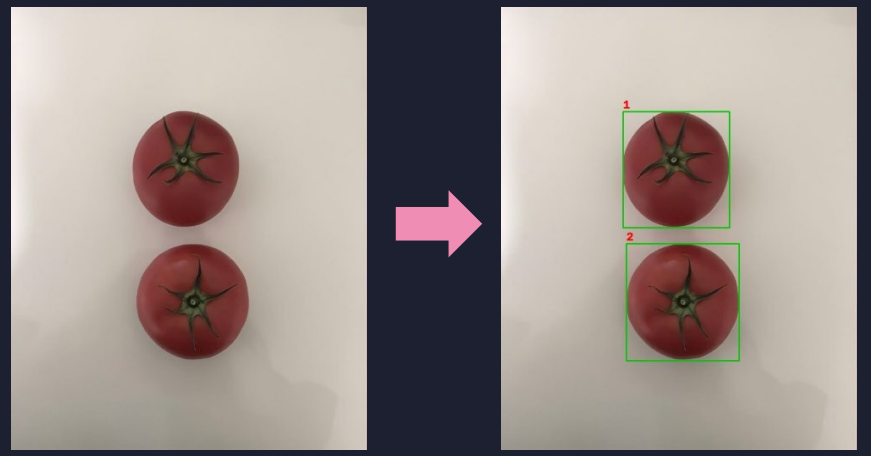
Đếm sản phẩm bằng xử lý ảnh

NHÓM 01

Trần Thị Kim Anh · 24741381
Phần 1: Morphology

Lương Ngô Gia An · 24630641
Phần 2: Connected Components

Trần Lý Hoàng Đức · 24746521
Phần 3: Thống kê & Giao diện



Đầu vào

Đầu ra: 2

Xám → Gauss → Otsu → Morphology → Connected Components → Thống kê

GIỚI THIỆU

Mục tiêu và quy trình

Morphology
làm sạch mask: xóa nhiễu, lấp lỗ

Connected Components
tách và đánh số từng vật

Thống kê
số lượng, diện tích, độ chính xác

Giao diện
mở ảnh/video, đổi tham số, lưu

Quy trình xử lý

Ảnh màu → Ảnh xám → Gauss 5×5 → Otsu → Morphology → Connected Components → Thống kê

Gauss 5×5: làm mờ để giảm nhiễu hạt trước khi tìm ngưỡng

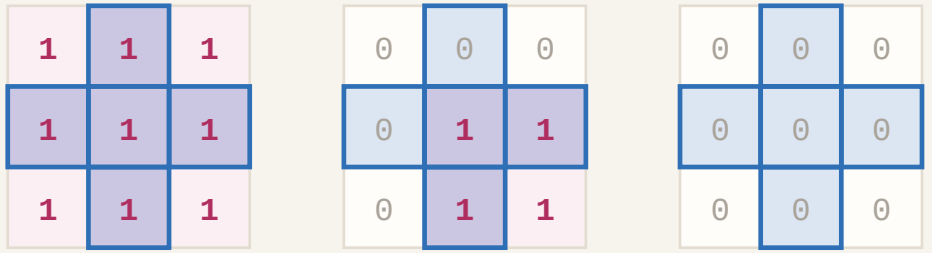
Otsu: thử mọi ngưỡng T = 0…255, chọn T làm $\sigma^2(T) = w_0 \cdot w_1 \cdot (\mu_0 - \mu_1)^2$ lớn nhất → vật trắng, nền đen

Ký hiệu chung

$$g[x, y] = \mathrm{OR}[\ \mathrm{SE}\{\ f[x, y]\ \}\ ] := \mathrm{dilate}(f, \mathrm{SE})$$
$$g[x, y] = \mathrm{AND}[\ \mathrm{SE}\{\ f[x, y]\ \}\ ] := \mathrm{erode}(f, \mathrm{SE})$$
$$\mathrm{open}(f, \mathrm{SE}) = \mathrm{dilate}(\ \mathrm{erode}(f, \mathrm{SE}), \mathrm{SE}\ )$$
$$\mathrm{close}(f, \mathrm{SE}) = \mathrm{erode}(\ \mathrm{dilate}(f, \mathrm{SE}), \mathrm{SE}\ )$$

f ảnh nhị phân (1 = vật, 0 = nền)
SE phần tử cấu trúc, chữ thập 3×3 (5 ô)
c số ô 1 của f nằm dưới SE
Giãn: HIT hoặc FIT → 1 **Co:** chỉ FIT → 1

Đặt SE lên f: 3 trường hợp



FIT	HIT	MISS
$c = 5$	$c = 3$	$c = 0$
SE nằm trọn trong vật	SE chạm vật một phần	SE không chạm vật

Nhận xét: giãn chỉ cần SE chạm vật (HIT), co cần SE nằm trọn trong vật (FIT).

Phép giãn (Dilation)

$$g[x, y] = \mathrm{OR}[\ \mathrm{SE}\{\ f[x, y]\ \}\ ] := \mathrm{dilate}(f, \mathrm{SE})$$

$$\mathrm{dilate}(f, \mathrm{SE}) = \theta(c, 1)$$

$$f \oplus \mathrm{SE} = \{\ z \mid (\mathrm{SE})_z \cap f \neq \emptyset\ \}$$

f (ảnh vào)

y\x	0	1	2	3	4
0	0	0	0	0	0
1	0	**1**	0	**1**	0
2	0	**1**	0	**1**	0
3	0	**1**	0	**1**	0
4	0	0	0	0	0

SE

0	**1**	0
1	**1**	**1**
0	**1**	0

Nhận xét: HIT hoặc FIT → 1. Vật nở ra, lấp khe hở.

Phép giãn · Bước tại [2, 2]

f SE tại [2, 2]

y\x	0	1	2	3	4
0	0	0	0	0	0
1	0	1	0	1	0
2	0	1	0	1	0
3	0	1	0	1	0
4	0	0	0	0	0

HIT

$$c = 2$$

MISS: c = 0
HIT: c = 1…4
FIT: c = 5

c (số ô 1 dưới SE) đang điền

y\x	0	1	2	3	4
0	0	1	0	1	0
1	1	2	2	2	1
2	1	3	2		
3					
4					

Nhận xét: giữa khe, chạm 2 thanh → HIT, c = 2.

Phép giãn · Đổi bảng c thành mask

f

y\x	0	1	2	3	4
0	0	0	0	0	0
1	0	1	0	1	0
2	0	1	0	1	0
3	0	1	0	1	0
4	0	0	0	0	0

đếm c →

c (số ô 1 dưới SE)

y\x	0	1	2	3	4
0	0	1	0	1	0
1	1	2	2	2	1
2	1	3	2	3	1
3	1	2	2	2	1
4	0	1	0	1	0

MISS HIT FIT

HIT/FIT → 1 →

f ⊕ SE mask

y\x	0	1	2	3	4
0	0	1	0	1	0
1	1	1	1	1	1
2	1	1	1	1	1
3	1	1	1	1	1
4	0	1	0	1	0

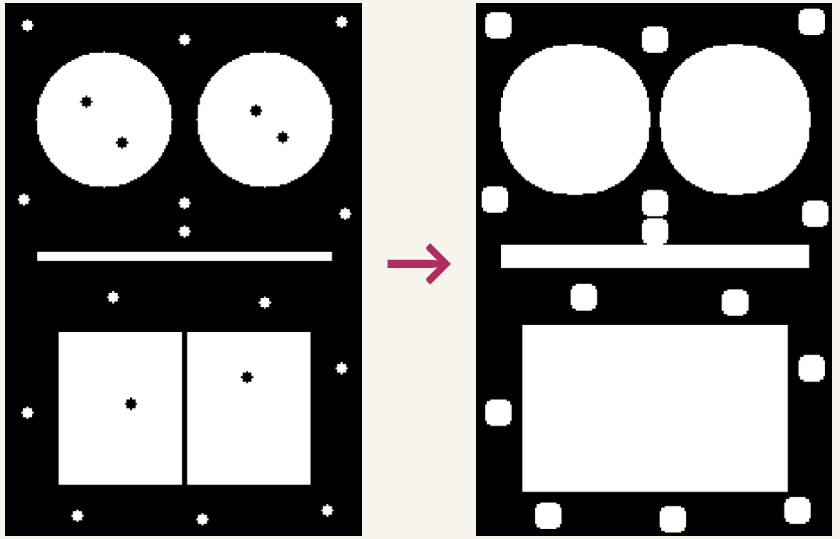
Nhận xét: mọi ô HIT/FIT thành 1 → khe được lấp, 2 thanh nối thành 1 khối.

Code chạy tay: phép giãn

```python
# DILATION - Phep gian
def dilate(mask, k):
    p = k // 2
    h, w = mask.shape
    pad = np.pad(mask, p, constant_values=0)
    ket_qua = np.zeros_like(mask)
    for y in range(h):
        for x in range(w):
            vung = pad[y:y+k, x:x+k]
            if np.any(vung == 255):  # θ(c, 1)
                ket_qua[y, x] = 255
    return ket_qua

# Ví dụ: ảnh test, k = 9
ket_qua = dilate(anh_test, 9)
```

Ví dụ: ảnh test, k = 9



Trước: ảnh test

Sau: phép giãn

Lỗ, vết nứt được lấp; nhiễu, sợi mảnh to ra

Phép co (Erosion)

$$g[x, y] = \mathrm{AND}[\ \mathrm{SE}\{\ f[x, y]\ \}\ ] := \mathrm{erode}(f, \mathrm{SE})$$

$$\mathrm{erode}(f, \mathrm{SE}) = \theta(c, S),\ \ S = 5$$

$$f \ominus \mathrm{SE} = \{\ z \mid (\mathrm{SE})_z \subseteq f\ \}$$

f (ảnh vào)

y\x	0	1	2	3	4
0	0	0	0	0	**1**
1	0	**1**	**1**	**1**	0
2	0	**1**	**1**	**1**	0
3	0	**1**	**1**	**1**	0
4	0	0	0	0	0

SE

0	**1**	0
1	**1**	**1**
0	**1**	0

Nhận xét: chỉ FIT → 1. Vật nhỏ lại, xóa nhiễu.

Phép co · Bước tại [2, 2]

f SE tại [2, 2]

y\x	0	1	2	3	4
0	0	0	0	0	1
1	0	1	1	1	0
2	0	1	1	1	0
3	0	1	1	1	0
4	0	0	0	0	0

FIT

$$c = 5$$

MISS: c = 0
HIT: c = 1…4
FIT: c = 5

c (số ô 1 dưới SE) đang điền

y\x	0	1	2	3	4
0	0	1	1	2	1
1	1	3	4	3	2
2	1	4	5		
3					
4					

Nhận xét: tâm khối, SE nằm trọn trong vật → FIT, c = 5.

Phép co · Đổi bảng c thành mask

f

y\x	0	1	2	3	4
0	0	0	0	0	1
1	0	1	1	1	0
2	0	1	1	1	0
3	0	1	1	1	0
4	0	0	0	0	0

đếm c →

c (số ô 1 dưới SE)

y\x	0	1	2	3	4
0	0	1	1	2	1
1	1	3	4	3	2
2	1	4	5	4	1
3	1	3	4	3	1
4	0	1	1	1	0

MISS HIT FIT

FIT → 1 →

f ⊖ SE mask

y\x	0	1	2	3	4
0	0	0	0	0	0
1	0	0	0	0	0
2	0	0	1	0	0
3	0	0	0	0	0
4	0	0	0	0	0

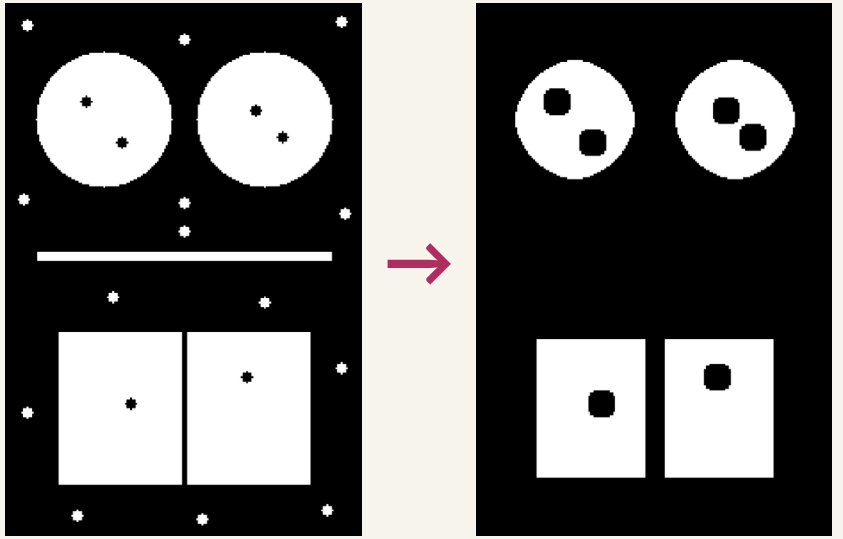
Nhận xét: chỉ ô FIT (c = 5) thành 1 → còn tâm khối, nhiễu bị xóa.

Code chạy tay: phép co

```python
# EROSION - Phep co
def erode(mask, k):
    p = k // 2
    h, w = mask.shape
    pad = np.pad(mask, p, constant_values=255)
    ket_qua = np.zeros_like(mask)
    for y in range(h):
        for x in range(w):
            vung = pad[y:y+k, x:x+k]
            if np.all(vung == 255):  # θ(c, S)
                ket_qua[y, x] = 255
    return ket_qua

# Ví dụ: ảnh test, k = 9
ket_qua = erode(anh_test, 9)
```

Ví dụ: ảnh test, k = 9



Trước: ảnh test

Sau: phép co

Nhiễu, sợi mảnh bị xóa; lỗ, vết nứt to ra

Phép mở (Opening)

$$\text{open}(f, SE) = \text{dilate}(\ \text{erode}(f, SE), SE\ )$$

$$f \circ SE = (f \ominus SE) \oplus SE$$

co trước → giãn sau

f (ảnh vào)

y\x	0	1	2	3	4
0	0	0	0	0	0
1	0	0	**1**	0	0
2	0	**1**	**1**	**1**	0
3	0	0	**1**	0	0
4	0	0	0	0	**1**

SE

0	**1**	0
1	**1**	**1**
0	**1**	0

Nhận xét: xóa nhiễu nhỏ hơn SE, vật lớn giữ hình dạng.

Phép mở · Tổng kết

f (ảnh vào)

y\x	0	1	2	3	4
0	0	0	0	0	0
1	0	0	**1**	0	0
2	0	**1**	**1**	**1**	0
3	0	0	**1**	0	0
4	0	0	0	0	**1**

co →

$g = f \ominus SE$

y\x	0	1	2	3	4
0	0	0	0	0	0
1	0	0	0	0	0
2	0	0	**1**	0	0
3	0	0	0	0	0
4	0	0	0	0	0

giãn →

$f \circ SE$

y\x	0	1	2	3	4
0	0	0	0	0	0
1	0	0	**1**	0	0
2	0	**1**	**1**	**1**	0
3	0	0	**1**	0	0
4	0	0	0	0	0

Nhận xét: chữ thập giữ nguyên, chỉ nhiễu mất (ô viền xanh).

Phép đóng (Closing)

$$\text{close}(f, SE) = \text{erode}(\ \text{dilate}(f, SE), SE\ )$$

$$f \bullet SE = (f \oplus SE) \ominus SE$$

giãn trước → co sau

f (ảnh vào)

y\x	0	1	2	3	4
0	0	0	0	0	0
1	0	**1**	**1**	0	0
2	0	**1**	0	**1**	0
3	0	0	**1**	**1**	0
4	0	0	0	0	0

SE

0	**1**	0
1	**1**	**1**
0	**1**	0

Nhận xét: lấp lỗ nhỏ, nối mảnh gần nhau, vật giữ kích thước.

Phép đóng · Tổng kết

f (ảnh vào)

y\x	0	1	2	3	4
0	0	0	0	0	0
1	0	1	1	0	0
2	0	1	0	1	0
3	0	0	1	1	0
4	0	0	0	0	0

giãn →

$g = f \oplus SE$

y\x	0	1	2	3	4
0	0	1	1	0	0
1	1	1	1	1	0
2	1	1	1	1	1
3	0	1	1	1	1
4	0	0	1	1	0

co →

$f \bullet SE$

y\x	0	1	2	3	4
0	0	0	0	0	0
1	0	1	1	0	0
2	0	1	1	1	0
3	0	0	1	1	0
4	0	0	0	0	0

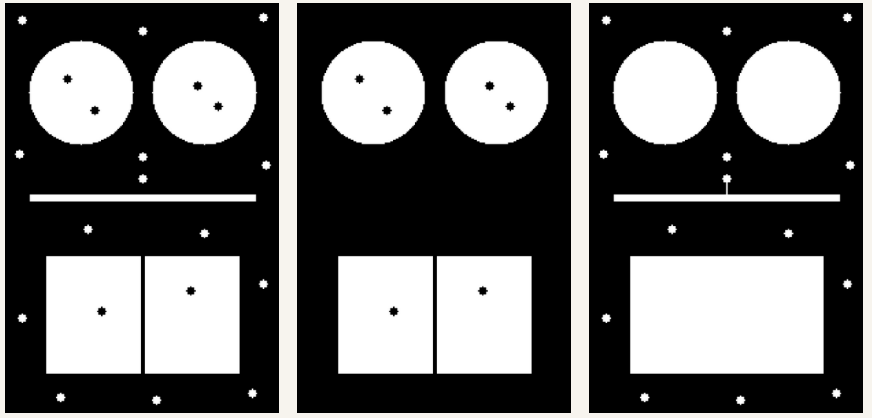
Nhận xét: vật giữ kích thước, chỉ ô lỗ được lấp (ô viền xanh).

Code chạy tay: phép mở, phép đóng

```python
# Phép mở: co trước, giãn sau
def opening(mask, k):
    return dilate(erode(mask, k), k)
# Phép đóng: giãn trước, co sau
def closing(mask, k):
    return erode(dilate(mask, k), k)

# File xử lý count.py: gọi OpenCV cho nhanh
kernel = cv2.getStructuringElement(
    cv2.MORPH_RECT, (k, k))
mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)
mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel)
```

Ví dụ: ảnh test, k = 9

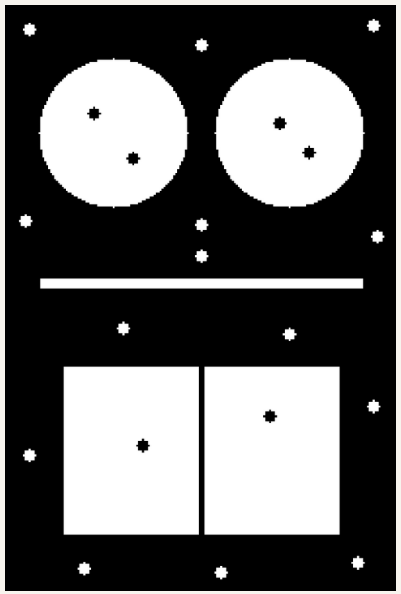


Ảnh test | Sau mở | Sau đóng

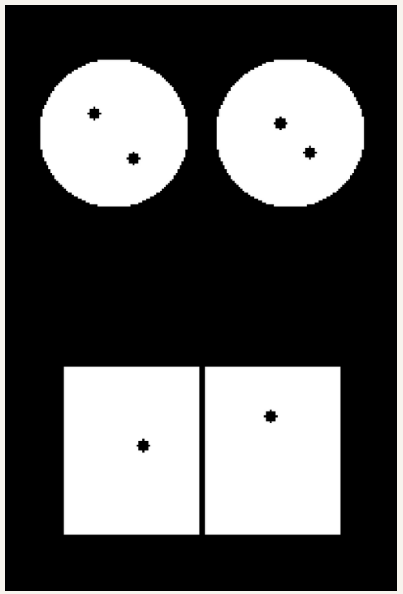
Mở: nhiễu, sợi mảnh mất; lỗ còn
Đóng: lỗ, vết nứt được lấp; nhiễu còn

Vì sao mặc định Opening + Closing

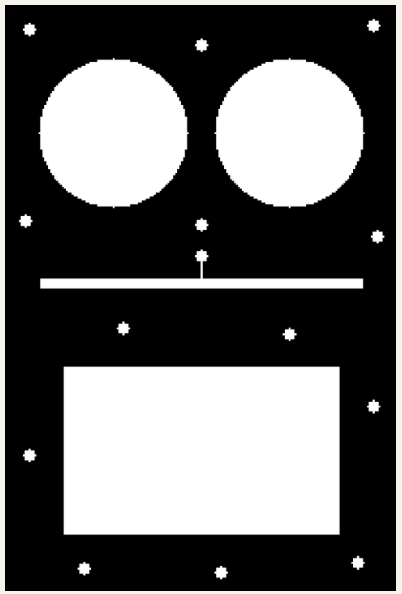
Cùng ảnh test, k = 9: đếm số vùng sau mỗi lựa chọn (vật thật: 3)



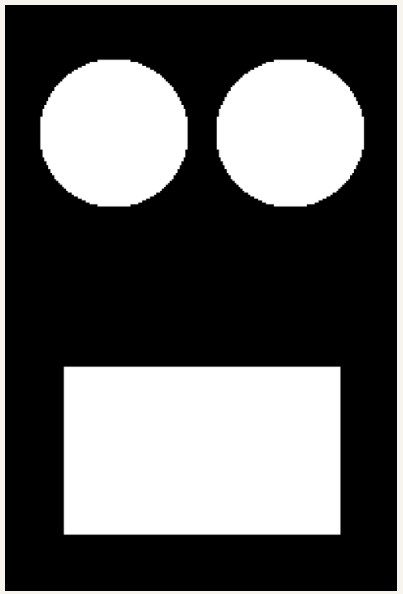
Ảnh test
19 vùng



Chỉ Opening
4 vùng ×



Chỉ Closing
17 vùng ×



Opening + Closing
3 vùng ✓

Vì sao

Chỉ mở: vết nứt còn. Chỉ đóng: nhiễu còn

Mở xóa nhiễu, đóng lấp lỗ → cần cả hai

Vẫn chỉ dùng co và giãn

Thứ tự: mở trước để xóa nhiễu, rồi đóng để lấp lỗ · trên giao diện vẫn chọn được từng phép riêng

Trước / sau và đánh giá

Ví dụ chung: ảnh đồng xu (7 xu)



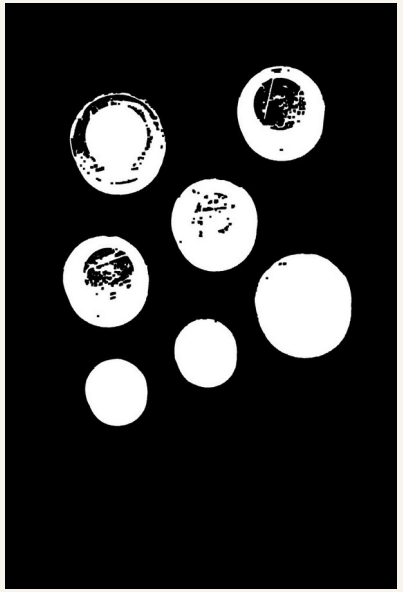
Ảnh gốc



Trước: mask Otsu
vỡ nhiều mảnh



Mặc định k = 3
vẫn vỡ ×



Closing k = 9
mỗi xu 1 khối ✓

Đánh giá

k = 3 nhỏ hơn khe giữa các mảnh → không nối được

Closing k = 9 lấp được khe → mask liền

Chưa đếm: đánh số làm ở phần 2

Tham số trên giao diện: kernel 3 / 5 / 7 / 9 · 6 phép toán · mặc định Opening + Closing, k = 3

Morphology: ưu và nhược điểm

Ưu điểm

- Mở xóa chấm nhiễu, đóng lấp lỗ và nối mảnh vỡ
- Cách làm đơn giản: chỉ xét ô k×k quanh mỗi điểm (min / max)
- Chỉnh được cỡ kernel và phép toán theo từng ảnh

Nhược điểm

- Chọn k sai thì hỏng: k nhỏ không nối được mảnh (đồng xu k = 3), k lớn làm dính các vật gần nhau
- Một cỡ k cho cả ảnh, không hợp vật to nhỏ khác nhau
- Code tay chạy chậm nên project dùng OpenCV

Lân cận 4 và lân cận 8

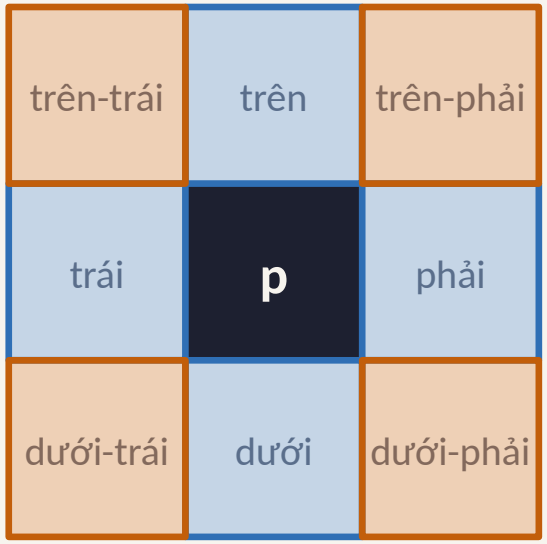
Lân cận của p

Các ô nằm sát điểm p (ô đen giữa lưới).

Liên thông

Hai điểm trắng liên thông nếu đi từ điểm này tới điểm kia được, mỗi bước sang một ô lân cận trắng.

→ Mỗi nhóm điểm liên thông là một vật.



1. **4-lân cận: 4 ô xanh kề cạnh p**
2. **8-lân cận: thêm 4 ô cam ở góc chéo**
3. **Tham số: chọn 4 hoặc 8, mặc định 8**

Gán nhãn từng bước

$$L(q) = L(p)$$
$$\text{nếu } q \text{ là lân cận của } p \text{ và } f(q) = 1$$

f: ảnh nhị phân sau Morphology
p: điểm đã có nhãn
q: điểm lân cận của p
L: ảnh nhãn, 0 là nền
Nhãn mới: gặp điểm 1 chưa có nhãn → nhãn + 1

Định nghĩa: mỗi vùng liên thông nhận một nhãn riêng; số nhãn = số vật.

f (ảnh vào)

1	1	0	0	0
1	1	0	0	1
1	1	0	0	1
0	0	0	1	1
0	1	0	0	0

L (ảnh nhãn)

1	1	·	·	·
1	1	·	·	2
1	1	·	·	2
·	·	·	2	2
·	3	·	·	·

① Quét từng ô: gặp ô 1 đầu tiên → nhãn 1

② Loang sang lân cận là 1 → cả khối nhãn 1

③ Gặp ô 1 chưa có nhãn → nhãn 2, loang tiếp

④ **Ô lẻ → nhãn 3. Hết ảnh: 3 vùng**

Ví dụ tay: 4 hay 8 lân cận

1	1	0	0	0
1	1	0	0	1
0	0	1	0	1
0	0	0	0	1
1	0	0	1	1

Ảnh B
5×5

1	1	·	·	·
1	1	·	·	2
·	·	3	·	2
·	·	·	·	2
4	·	·	2	2

4-lân cận
4 vùng

1	1	·	·	·
1	1	·	·	2
·	·	1	·	2
·	·	·	·	2
3	·	·	2	2

8-lân cận
3 vùng

Ô (2, 2) chỉ chạm khối trên-trái theo **đường chéo** → 4-lân cận tách riêng, 8-lân cận gộp vào nhãn 1.

Code chạy tay: gán nhãn bằng loang

```python
def gan_nhan(mask, lan_can):
    h, w = mask.shape
    nhan = np.zeros((h, w), dtype=int)
    huong = [(-1, 0), (1, 0), (0, -1), (0, 1)]        # 4 hướng
    if lan_can == 8:
        huong += [(-1, -1), (-1, 1), (1, -1), (1, 1)] # chéo
    so_nhan = 0
    for y in range(h):
        for x in range(w):
            if mask[y, x] == 255 and nhan[y, x] == 0:
                so_nhan += 1              # nhãn mới
                nhan[y, x] = so_nhan
                cho_loang = [(y, x)]
                while cho_loang:          # loang
                    cy, cx = cho_loang.pop()
                    for dy, dx in huong:
                        ny, nx = cy + dy, cx + dx
                        if 0 <= ny < h and 0 <= nx < w:
                            if mask[ny, nx] == 255 and nhan[ny, nx] == 0:
                                nhan[ny, nx] = so_nhan
                                cho_loang.append((ny, nx))
    return so_nhan, nhan
```

① huong
= 4 hướng, chọn 8 thì thêm 4 hướng chéo

② Quét từng điểm
gặp ô trắng chưa có nhãn → nhãn mới

③ Loang
lân cận trắng, chưa có nhãn → cùng nhãn

④ Ví dụ kẹo trái tim
so_nhan = 5 vùng, giống OpenCV



Code trong project: file xử lý count.py

File xử lý ảnh: count.py

```
# 4. Connected Components (4 hoặc 8 lân cận)
n, labels, stats, centers = cv2.connectedComponentsWithStats(
    mask, connectivity=connectivity)
for i in range(1, n):                    # nhãn 0 là nền
    x, y, w, h, area = stats[i]     # khung + diện tích
    if area < min_area:
        continue
    dien_tich.append(int(area))
    cv2.rectangle(ket_qua, (x, y), (x + w, y + h),
                  (0, 200, 0), do_day)
anh_nhan = to_mau_nhan(labels, n)   # mỗi nhãn một màu
```

Tham số

Liên thông: 4 / 8

Mặc định: 8

Trả về

n: số nhãn

labels: ảnh nhãn

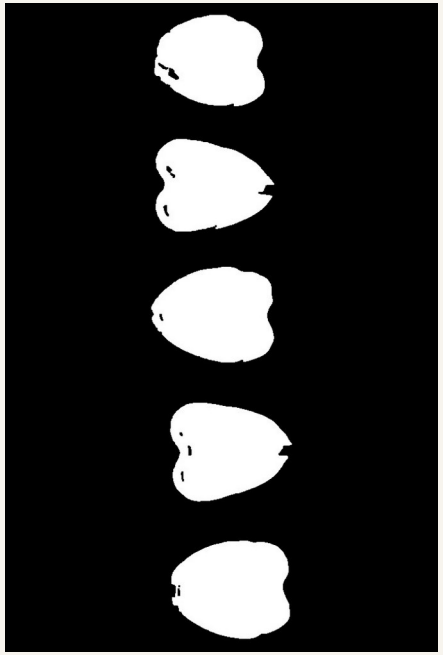
stats: x, y, w, h, diện tích

Ví dụ kẹo trái tim

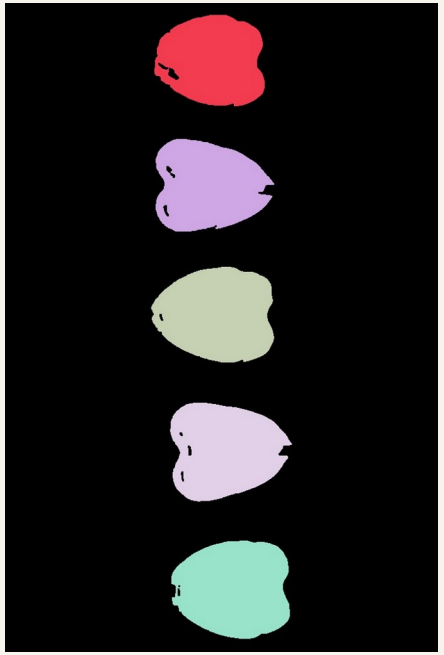
n − 1 = 5 vùng, giống code tay

Trước / sau và đánh giá

Ví dụ chung: ảnh kẹo trái tim (5 viên)



Trước: mask
sau Morphology



Sau: ảnh nhãn
5 vùng

Đánh giá

5 vùng = 5 viên kẹo, khớp thực tế ✓

Mỗi viên một nhãn, một màu riêng

Đổi tham số

8-lân cận: 5 vùng

4-lân cận: 5 vùng

Như nhau vì viền kẹo liền, không có chỗ chỉ chạm chéo

Connected Components: ưu và nhược điểm

Ưu điểm

- Tự đếm: mỗi vùng một nhãn là một vật
- Có sẵn diện tích và khung của từng vật
- Chọn được 4 hoặc 8 lân cận theo ảnh

Nhược điểm

- Chỉ đánh số theo mask: mask vỡ thì 1 vật thành nhiều nhãn
- 2 vật chạm nhau → nối liền, chỉ được 1 nhãn
- Vùng trắng nào cũng đánh số, kể cả vết bẩn → phải lọc ở phần 3

Ví dụ nhược điểm 1 · ảnh cúc áo, phóng to 1 chiếc cúc



1 chiếc cúc → mask: 2 mảnh → 2 nhãn

Cúc màu kem gần màu nền → mask tách 2 mảnh → **2 nhãn, đếm dư 1**. Connected Components đánh số đúng theo mask, lỗi nằm ở mask.

PHẦN 3 · THỐNG KÊ

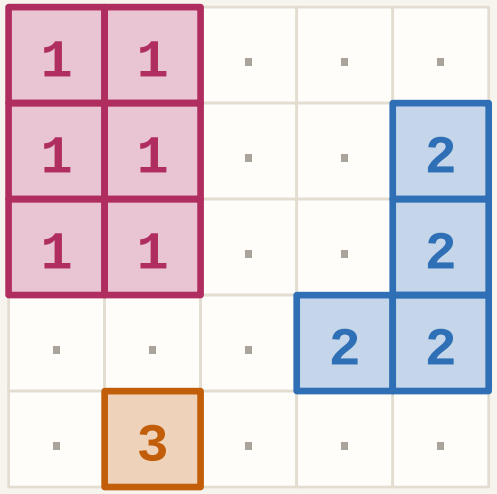
Diện tích và lọc vùng nhỏ

$$A_i = \text{số điểm có } L = i$$
$$\text{giữ vùng } i \text{ nếu } A_i \geq \text{min_area}$$

L: ảnh nhãn từ Connected Components
i: số nhãn (1, 2, 3, ...)
A_i: diện tích vùng i, đơn vị px
min_area: tham số, ô nhập trên giao diện
N: số vùng được giữ = số sản phẩm

Định nghĩa: diện tích là số điểm ảnh của vùng; vùng nhỏ hơn min_area coi là bụi.

L (ảnh nhãn)



min_area = 2

A_1 = 6 giữ ✓

A_2 = 4 giữ ✓

A_3 = 1 bỏ ×

N = 2

① Nhãn 1: đếm được 6 ô → $A_1 = 6 \geq 2$, giữ

② Nhãn 2: 4 ô → $A_2 = 4 \geq 2$, giữ

③ Nhãn 3: 1 ô → $A_3 = 1 < 2$, bỏ (bụi)

Kết quả: N = 2 sản phẩm

Tổng, trung bình và độ lệch chuẩn

$$S = \Sigma A_i \qquad \bar{A} = S / N$$
$$\sigma = \sqrt{(\Sigma (A_i - \bar{A})^2 / N)}$$

S: tổng diện tích các vùng được giữ
Ā: diện tích trung bình một vật
min, max: vật nhỏ nhất, lớn nhất
σ: độ lệch chuẩn, các vật đều cỡ thì σ nhỏ
Σ: cộng qua mọi vùng i = 1 … N

Định nghĩa: mô tả kích thước sản phẩm: tổng, trung bình, nhỏ nhất, lớn nhất và độ đều.

Từ slide trước: **$A_1 = 6$, $A_2 = 4$, $N = 2$**

1. **Tổng:** $S = 6 + 4 = 10$
2. **Trung bình:** $\bar{A} = 10 / 2 = 5$
3. **Nhỏ nhất, lớn nhất:** 4 và 6
4. **Độ lệch chuẩn:** $\sigma = \sqrt{((1^2 + 1^2) / 2)} = 1$

σ = 1 nhỏ → hai vật gần bằng cỡ nhau

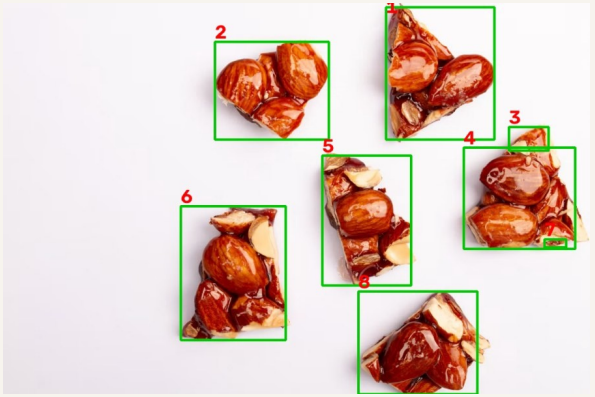
PHẦN 3 · THỐNG KÊ

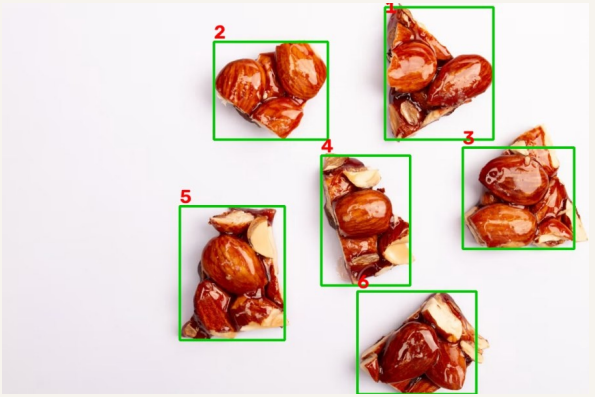
Đánh giá: độ chính xác

$$\text{độ chính xác} = \max(0,\ 1 - |N - T| \ /\ T)$$

N: số sản phẩm chương trình đếm
T: số thực tế, người dùng nhập
|N − T|: số vật đếm lệch
max(0, ...): không để kết quả âm

Định nghĩa: số đếm đúng được bao nhiêu phần trăm so với số thật.





① T = 6, min_area 300: N = 8 **1 − |8 − 6| / 6 = 66,7%**

② min_area 2000: N = 6 **1 − 0 / 6 = 100%**

③ Nếu đếm lệch nhiều, vd N = 13 **1 − 7 / 6 < 0 → lấy 0%**

Trung bình 10 ảnh = cộng 10 độ chính xác rồi chia 10

Code chạy tay: tính thống kê bằng vòng lặp

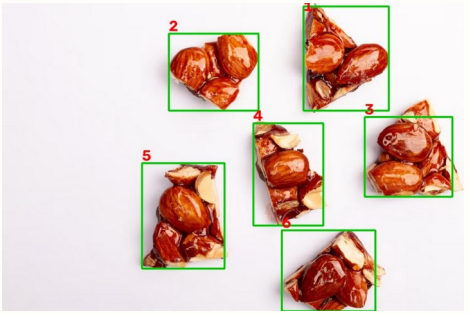
```python
def thong_ke_tay(dien_tich):
    n = len(dien_tich)
    tong = 0
    for a in dien_tich:
        tong += a                       # S = ΣAᵢ
    tb = tong / n                       # Ā = S / N
    nho = dien_tich[0]
    lon = dien_tich[0]
    for a in dien_tich:
        if a < nho:
            nho = a
        if a > lon:
            lon = a
    tong_bp = 0
    for a in dien_tich:
        tong_bp += (a - tb) ** 2        # Σ(Aᵢ − Ā)²
    lech = (tong_bp / n) ** 0.5         # σ
    return tong, tb, nho, lon, lech
```

① Vòng 1
cộng dồn → S, chia n → Ā

② Vòng 2
so từng số → nhỏ nhất, lớn nhất

③ Vòng 3
cộng $(A_i - Ā)^2$, chia n, lấy căn → σ

④ Ví dụ hạnh nhân
S = 157.730, Ā ≈ 26.288,3, giống numpy



6 miếng
min 2000

Code trong project: file xử lý và file giao diện

```
# File xử lý (count.py): lọc vùng nhỏ, lưu diện tích
for i in range(1, n):
    x, y, w, h, area = stats[i]
    if area < min_area:
        continue
    dien_tich.append(int(area))          # Aᵢ

def thong_ke(dien_tich):
    a = np.array(dien_tich)
    return (int(a.sum()), float(a.mean()),   # S, Ā
            int(a.min()), int(a.max()), float(a.std()))

# File giao diện (count_gui.py): tính độ chính xác
do_chinh_xac = max(0, 1 - abs(count - t) / t) * 100
```

Tham số

Diện tích nhỏ nhất (min_area): ô nhập số

Mặc định: 300

Đánh giá

Ô nhập số thực tế T

Ví dụ hạnh nhân (min 2000)

S = 157.730, Ā ≈ 26.288,3

min 23.290, max 31.650, σ ≈ 2.761

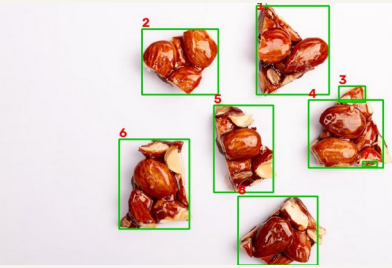
giống code tay

PHẦN 3 · THỐNG KÊ

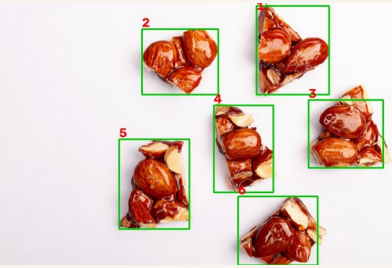
Trước / sau và đánh giá



Ảnh gốc
6 miếng



min_area 300
đếm 8 ×



min_area 2000
đếm 6 ✓

Ảnh	Thực tế	Mặc định	Chỉnh	Tham số chỉnh
Cà chua	2	2 ✓	2 ✓	–
Cá vàng	3	3 ✓	3 ✓	–
Bánh gấu	9	9 ✓	9 ✓	–
Chữ cái	25	25 ✓	25 ✓	–
Trái tim	5	5 ✓	5 ✓	–
Cúc áo	32	34 ×	32 ✓	Closing k=7
Hạnh nhân	6	8 ×	6 ✓	min_area 2000
Kẹo que	29	33 ×	29 ✓	kernel 7
Đồng xu	7	18 ×	7 ✓	Closing k=9
Nền tối	12	1 ×	12 ✓	vật sáng hơn nền

Độ chính xác TB trên 10 ảnh: **75,5%** (mặc định) → **100%** (chỉnh tham số)

Thống kê: ưu và nhược điểm

Ưu điểm

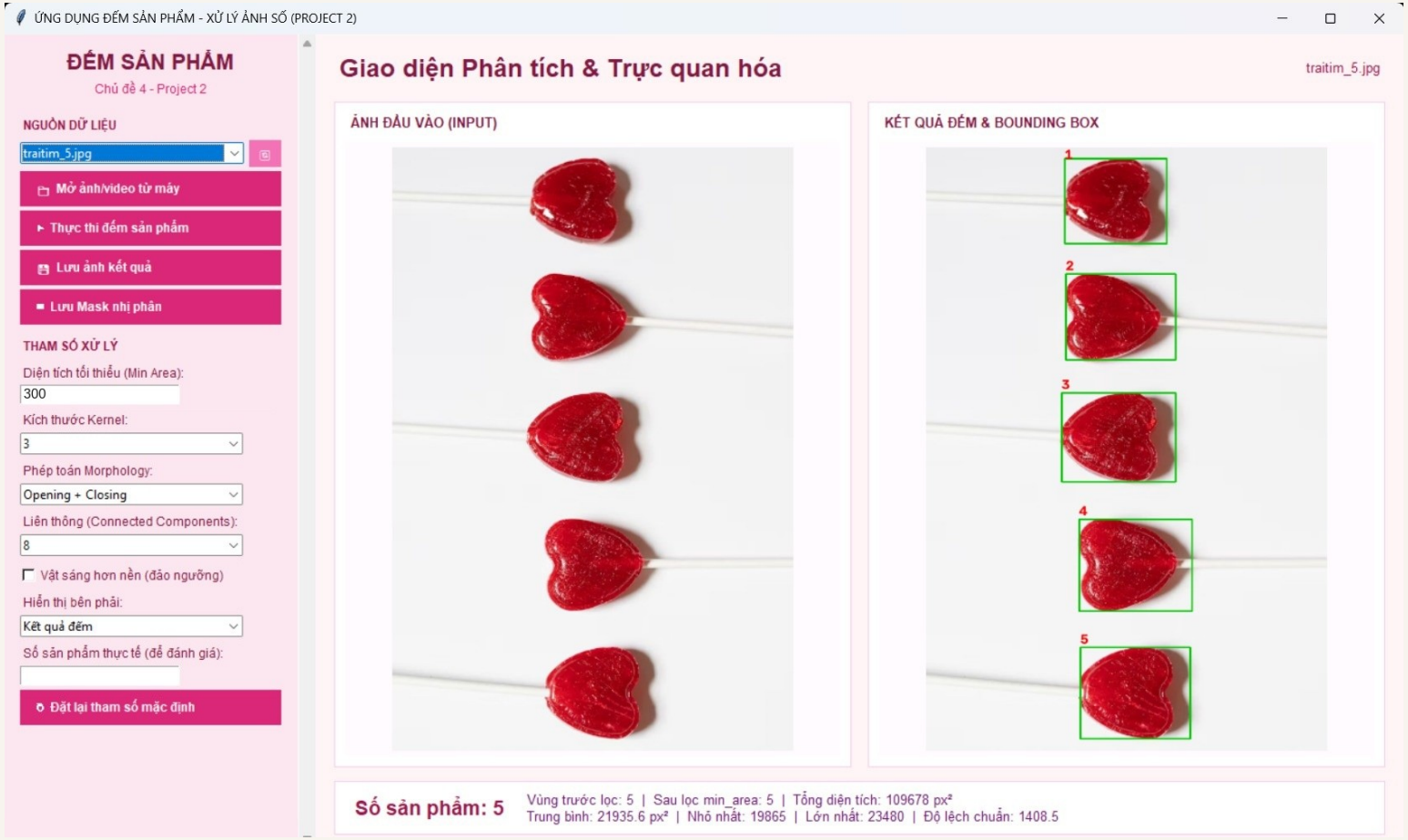
- Lọc mảnh vụn bằng diện tích nhỏ nhất (min_area)
- Cho biết kích thước vật: tổng, trung bình, nhỏ nhất, lớn nhất, độ lệch chuẩn
- Có độ chính xác để đánh giá kết quả bằng con số

Nhược điểm

- Diện tích nhỏ nhất phải chọn theo từng ảnh (hạnh nhân: 300 sai, 2000 đúng)
- Vật thật nhỏ hơn mức này cũng bị bỏ nhầm
- Phải biết số thực tế T mới tính được độ chính xác

PHẦN 3 · GIAO DIỆN

Mở ảnh/video, đổi tham số, lưu kết quả



Mở ảnh / video
dataset hoặc từ máy

Đổi tham số
tự cập nhật kết quả

Xem trước / sau
4 chế độ hiển thị

Lưu kết quả
ảnh kết quả và mask

TỔNG KẾT

Kết luận và hướng phát triển

Kết luận

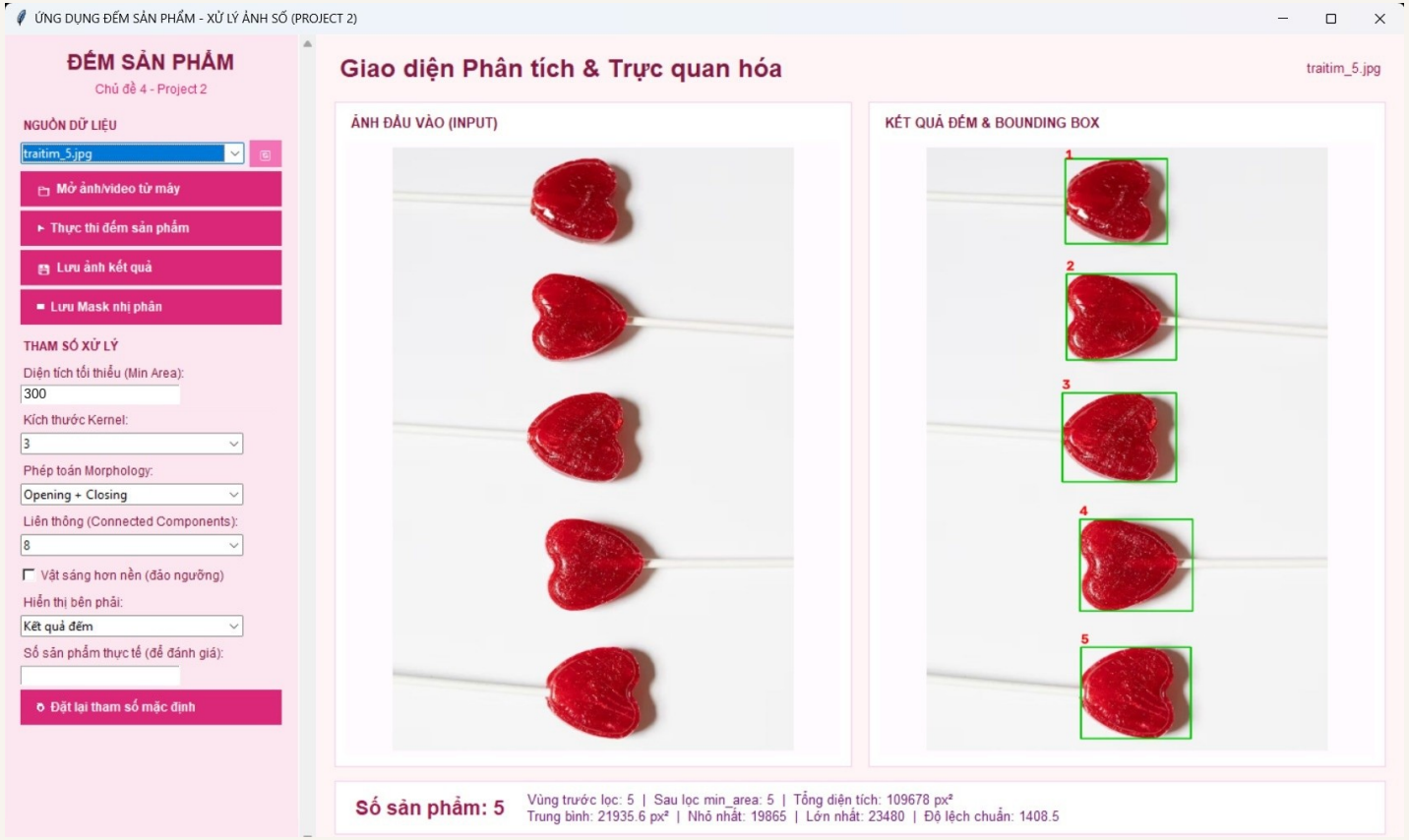
- Mặc định: đúng 5/10 ảnh, độ chính xác TB 75,5%
- Chỉnh tham số: đúng 10/10 ảnh, 100%
- Không có bộ tham số chung cho mọi ảnh → giao diện cho chỉnh và xem trước/sau là cần thiết

Hướng phát triển

- Watershed để tách vật chạm nhau
- Ngưỡng thích nghi cho ảnh nền sáng không đều
- Tự gợi ý diện tích nhỏ nhất từ phân bố diện tích
- Đếm trên toàn bộ video, từng khung hình

DEMO

Demo chương trình



① Đồng xu

Closing k = 9 → xem mask trước / sau

② Kẹo trái tim

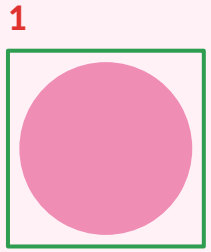
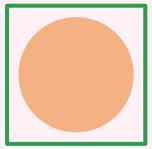
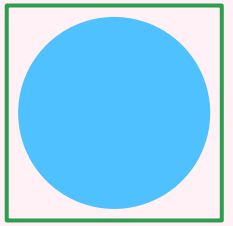
ảnh nhãn, đổi 4 / 8 lân cận

③ Hạnh nhân

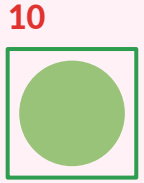
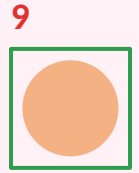
min_area 300 → 2000, nhập T = 6

④ Lưu

ảnh kết quả và mask


Cảm ơn thầy và các bạn đã lắng nghe!

Nhóm 01 · Chủ đề 4 – Project 2: Đếm sản phẩm

Trần Thị Kim Anh · Lương Ngô Gia An · Trần Lý Hoàng Đức

